

Pan-European University, Faculty of Informatics

Interim Research Report

Definition of inheritance and interface for object-centric processes

of the project

**Requirements and formal definition of a low-code
language based on object-centric processes -
LowcodeOCP**

Authors:

Gabriel Juhás, Milan Mladoniczky, Juraj Mažári, Tomáš Kováčik, Jakub
Kovář and Robin Bergenthum

Contact:

gabriel.juhas@paneurouni.com, milan.mladoniczky@paneurouni.com

Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia
under the project No.

09I03-03-V04-00493

August 31, 2026

Version: v0.1

Abstract

Inheritance is a key concept in object-oriented design and modelling. However, it remains largely absent in workflow modelling, where process behaviour is specified by prescriptive, executable process models. While existing research provides various theoretical notions of process inheritance, refinement, and behavioural substitutability, these concepts have not yet been integrated into workflow modelling languages used in practical model-driven workflow engines. This paper investigates possible notions of inheritance for process-centric objects, a paradigm that combines executable process models with object-oriented concepts. Following existing research, we identify two kinds of inheritance, derive requirements for their implementation in the process-centric objects paradigm based on existing theoretical approaches and practical experience, and discuss how the existing body of knowledge addresses the identified requirements.

Contents

Contents	i
1 Introduction	1
2 Related Work	3
3 Process-Centric Objects	5
4 Inheritance in Process-Centric Objects	7
4.1 Implementation inheritance	7
4.2 Type inheritance	9
5 Conclusion	12

Chapter 1

Introduction

In object-oriented design, inheritance is an established concept for structuring and relating models at different levels of abstraction. It allows common properties and behaviour to be captured once in a more general model and specialized in derived models. Inheritance additionally establishes an *is-a* relation that enables instances of specialized models to be used in place of instances of more general ones. These principles are most prominently embodied in object-oriented modelling and programming, where inheritance from classes and interfaces constitutes one of the central mechanisms for structuring complex systems [Tai96].

In the domain of model-driven workflow systems, the systems' behaviour is specified by executable process models and interpreted by workflow engines at runtime. Although existing workflow modelling languages used by the engines provide mechanisms for process composition and reuse, an inheritance mechanism is rarely available. Consequently, workflow modelling languages benefit only to a limited extent from the abstraction, specialization, and reuse enabled by inheritance in object-oriented modelling. At the same time, inheritance of process behaviour is not a new theoretical problem. Existing research has investigated several notions of process inheritance, refinement, and behavioural substitutability [GG01; AB97; Bv01; Nie93; NP90; BDP99]. These approaches provide theoretical foundations for relating general and specialized process models. However, the workflow modelling languages used in existing model-driven workflow engines, like Camunda, Flowable, jBPM, or Bonita BPM, do not yet make use of them.

The paradigm of *process-centric objects* provides a particularly promising setting for bridging this gap. The paradigm emerged to specialize on and simplify the design and implementation of business processes supported by information systems [Juh+22; Juh+24]. It combines executable process models with concepts from object-oriented programming, reactive programming, and relational databases. Process-centric objects therefore combine object-oriented modelling with executable workflow models. This makes the paradigm a suitable setting in which to investigate how an inheritance mechanism can be introduced into executable process modelling in practice.

In this paper, we investigate inheritance mechanisms in the process-centric objects paradigm and use the programming and modelling language *Petriflow* [Juh+25a; Juh+25b], which adheres to the paradigm, as a concrete practical setting for deriving its requirements. We review existing literature on inheritance notions for processes and process models. Based on these theoretical foundations and our practical experience with Petriflow [JMP24; Juh+24], we formulate requirements for an inheritance notion in the domain of process-centric objects and discuss how existing approaches can contribute to satisfying them. Beyond process-centric objects, the resulting concepts may also provide insights into how inheritance could be supported in model-driven workflow systems in general.

Chapter 2

Related Work

In this section, we provide a short overview of existing approaches studying inheritance notions, refinement, and behavioural substitutability of processes and process models.

Rob van Glabbeek and Ursula Goltz in their work *Refinement of actions and equivalence notions for concurrent systems* [GG01] study an operator for action refinement in the domain of concurrent systems design. This operator replaces one or more atomic activities (actions) in a process model on one level of abstraction with a more complicated subprocess on a lower level. The behaviour of the refined process is obtained as a combination of the behaviour of the more abstract process and the behaviour of the substituted subprocess. They also identify a subset of existing behavioural equivalence relations for processes that are preserved under their refinement.

Twan Basten and Wil van der Aalst [AB97; Bv01] study *Inheritance of behaviour* to facilitate model reuse in object-oriented design and modelling. They are interested in an inheritance relation between process models that allows a process playing the role of a subclass to enrich the behaviour of a superclass process with new activities. They find it is possible to define inheritance as a behavioural equivalence between the subclass and the superclass after the new activities of the subclass are hidden and/or blocked.

Oscar Nierstrasz in his work *Regular types for active objects* [Nie93] studies inheritance of behaviour from a type theoretical perspective. His *active objects* are processes, he views *type* as a *partial specification of behaviour*, and he proposes a type system that allows safe substitution of subtypes for supertypes in a system of communicating processes. The type of a process encodes the messages it can send and receive as well as the causal dependencies between them. A process is a subtype of (inherits from) another if it can participate on all message exchanges the supertype can without introducing unexpected failures.

Claudia Balzarotti, Fiorella De Cindio, and Lucia Pomello study in *Observation equivalences for the semantics of inheritance* [BDP99] inheritance in concurrent object-oriented languages from two perspectives. For inheritance as an *is-a* relation between processes, they follow along similar lines as Nierstrasz [Nie93] and Basten and van der

Aalst [AB97; Bv01], focusing on exhibited behaviour of processes. For inheritance as a *code reuse* mechanism, they focus on observable states of processes. They propose a semantic characterization of both kinds of inheritance based on preorder relations, as they believe these are better suited for modelling process extensions than equivalence relations.

Altogether, these approaches establish a theoretical foundation for inheriting process behaviour from different perspectives and with different motivations. However, their usage in modelling languages for practical model-driven workflow engines remains limited. We therefore investigate their applicability to the process-centric objects paradigm, which combines executable process models with object-oriented structures, data, and interactions.

Chapter 3

Process-Centric Objects

The paradigm of process-centric objects combines concepts from object-oriented programming, business process modelling, reactive programming, and relational databases and is based on four core pillars [Juh+22; Juh+25b]. This section introduces the paradigm and highlights the aspects that distinguish it from conventional business process modelling.

The first pillar is its abstraction of a full-stack software architecture. The presentation layer, the application layer and the database layer are made fully transparent by utilizing *vertically integrating objects*. Objects and their data attributes are defined only once, not for each layer separately, and the transfer of data between the layers is handled automatically by the interpreting engine [Juh+22].

Second, modelling with process-centric objects brings data, its manipulation and its evolution into the foreground by enriching objects with explicit lifecycle in the form of a prescriptive workflow process model. The workflow process defines activities of the object, associates them with data attributes, and defines causal dependencies between the activities. The currently enabled activities of each object are displayed in a user interface and when a user performs an activity, they can read or write the object's data attributes associated with the activity. Activities can be finished, progressing the workflow and changing the set of currently enabled activities accordingly [Juh+22].

Third, each operation in the system, such as starting an activity, finishing an activity, creating a new object, or changing a data attribute's value, emits an event which can be listened and reacted to. The reactions to events are specified in a domain-specific textual programming language and can perform computations, call external services, and most importantly, can trigger additional events by, for example, manipulating data attributes of objects, starting and finishing activities, or by creating new objects [Jur18].

The final fourth pillar of the process-centric objects paradigm is object discovery. Traditionally, in object-oriented programming, an object can obtain a reference to another object by either creating it, or by receiving the reference as an argument from one of its methods. The paradigm of process-centric objects allows a third option: an object can query the runtime environment to find other objects. Objects can be queried based

on their state, which is composed of their data attribute values and the values of state variables governing the lifecycle. This allows coevolving objects to create new relations between each other on the fly without prior knowledge of each other [Juh+22].

In summary, the process-centric objects paradigm allows us to implement an information system by identifying the objects that appear within it, modelling their lifecycle with a prescriptive workflow model, specifying their data attributes, associating the data attributes with the activities of the workflow, and specifying the reactions to the events produced by the object. A set of these process-centric workflow models is then interpreted by a runtime environment, which allows users to interact with the system, to instantiate the models into objects, and to view and perform the enabled activities of existing objects [Juh+22].

The combination of executable workflow models and object-oriented concepts makes the process-centric objects paradigm a natural setting for investigating inheritance in executable process modelling. Inheritance in this setting must account not only for process behaviour, but also for the data, state, and interactions of process-centric objects. This broader perspective allows us to study how theoretical notions of process inheritance can be transferred into a practical model-driven environment.

We chose Petriflow as the concrete practical basis for this investigation. Petriflow is a programming language adhering to the process-centric objects paradigm [Juh+25a; Juh+25b], and our experience using it to implement multiple large real-world information systems in a commercial setting [JMP24; Juh+24] revealed recurring situations demanding some form of inheritance. These situations provide the use cases from which we derive the requirements discussed in the following section.

Based on these practical requirements and the distinctions identified in the literature, we argue that inheritance for process-centric objects must address two conceptually different concerns. Following Balzarotti, De Cindio, and Pomello [BDP99], we distinguish *implementation inheritance*, which provides a mechanism for code reuse and specialization, and *type inheritance*, which establishes an *is-a* relation as a basis for substitutability. This suggests, that unlike in object-oriented programming and modelling, where inheritance is strictly an *is-a* relation and only incidentally permits a limited form of code reuse [Tai96], in the process-centric objects paradigm there may be two orthogonal independent kinds of inheritance. Investigating these mechanisms in process-centric objects may therefore not only extend the paradigm itself, but also provide insights into how inheritance can become a useful modelling mechanism in model-driven workflow systems in general.

Chapter 4

Inheritance in Process-Centric Objects

The requirements for an inheritance notion for process-centric object models arise from two complementary perspectives. On the one hand, models should be reusable and extensible so that shared data and process structures do not need to be duplicated. On the other hand, process-centric objects participate in a modular system of interacting processes, which requires a notion of substitutability that ensures that one process-centric object can safely replace another. We discuss these two concerns separately as *implementation inheritance* and *type inheritance*.

4.1 Implementation inheritance

We first motivate *implementation inheritance* through two examples from the health insurance domain encountered while implementing real-world systems in Petriflow. We then derive important properties that an *implementation inheritance* mechanism should provide and relate these requirements to approaches from the existing literature.

In health insurance, there are treatments and medical procedures that are not automatically covered by the insurance and instead are covered on a case-by-case basis. A patient that needs to undergo such a treatment must apply for reimbursement at the health insurance company. The *application for reimbursement* is a process-centric object. It has a lifecycle that starts with the patient filling in data relevant to the treatment, followed by submission to the health insurance company, assessment of the application, an approval decision, and eventually by payment for the treatment.

In the first example, the information a patient must provide depends on the treatment they need. We can think of each piece of information as an input field on an application form. Each field is modeled by a data attribute of the process-centric object. Depending on the specific treatment, a patient must fill out tens, or in some cases hundreds, of data

attributes. From a programming perspective, it is desirable to be able to specify one *abstract* workflow model that defines the shared lifecycle and the shared data attributes and then to be able to derive workflow models for each treatment by inheriting from the abstract model and extending it with the treatment-specific data attributes and their associations to activities.

In the first example, we make no modifications to the lifecycle. We only introduce new data attributes and associate them with existing activities. We call the workflow model playing the role of a "superclass" *abstract* because, like in object-oriented programming, it should not be instantiated. However, even the simple example opens important questions. A data association between a data attribute and an activity of a process-centric object is a rich relation. It, for example, specifies whether a user performing the activity is allowed to only view or also to change the data attribute's value. The question is therefore, whether we are allowed to modify the inherited data associations specified in the superclass. Since the goal of *implementation inheritance* is code reuse, we believe no change to the inherited data attributes, nor to their data associations should be permitted. In this respect, *implementation inheritance* should behave no-variantly.

In the second example, we focus on the approval part of our *application for reimbursement*. It may be the case that, depending on the treatment the patient needs to undergo, the approval is decided by a single employee, or it may be more complex, involving an entire subprocess and multiple people. An inheritance mechanism that facilitates code reuse should support this use case and allow us to extend and/or modify the lifecycle specified by a superclass's workflow model.

Implementation inheritance is studied by van Glabbeek and Goltz [GG01] and Balzarotti, De Cindio, and Pomello [BDP99], each from its own unique perspective. The approach of van Glabbeek and Goltz [GG01] gives more flexibility in extending the superclass's behaviour. They define a refinement operator, which allows us to replace a part of the superclass's workflow model with a new workflow fragment. The refinement operator is such, that it allows us to derive the behaviour of the subclass as a composition of the behaviour of the superclass, and the new workflow fragment. Furthermore, the authors express that a refinement operation in general should be such that, if we have two equivalent superclass workflow models and extend them in the same way, the resulting two subclass workflow models should remain equivalent. Finally, the authors study their operator in this regard and find which semantics equivalences are, and are not, preserved under refinement [GG01].

Balzarotti, De Cindio, and Pomello approach the topic of *implementation inheritance* from a different angle [BDP99]. They propose an inheritance relation between a subclass and its superclass should be a preorder relation. Roughly speaking, the subclass should include the behaviour of the superclass, but it may also introduce additional behaviour. To formalise *implementation inheritance*, they propose that not activities of objects, but

instead their states should play the deciding role. In their framework, the states of a workflow are divided between observable and unobservable. A workflow may introduce new activities and new unobservable states and it inherits from a superclass workflow if it can reproduce its observable state changes [BDP99].

Both proposals fit well with our second motivational example. An abstract *application for reimbursement* can move from a *pending approval* state to either an *approved* state or a *not approved* state. We do not want to specify whether this can be accomplished by a single activity or a longer subprocess, only that these state changes must be possible which can be achieved by refining the workflow in-between these states.

Overall, *implementation inheritance* in the process-centric objects paradigm should aim to alleviate two pain points. First, it should allow us to specify data attributes in a superclass and inherit them, alongside their associations to activities, into subclasses to avoid duplicities and aid in long term maintainability. Second, it should allow us to specify an abstract workflow, which we can refine, while maintaining its general "shape" and intention. In our opinion, we should look at how traditional object-oriented approaches inherit data to solve the first requirement and to approaches along the lines of van Glabbeek and Goltz [GG01] and Balzarotti, De Cindio, and Pomello [BDP99] to solve the second requirement.

4.2 Type inheritance

If we can understand an object-oriented program as a system of communicating objects [NP90], then we can understand a process-centric objects program as a system of communicating processes. The main motivation for *type inheritance* is *substitutability* [LW94]. We want to be able to replace individual (process-centric) objects in a system without breaking it.

Type systems in the object-oriented world abstract from internal states of objects and focus on the methods an object has and on their signatures. One object is a subtype of another if the signatures of all its methods are compatible [LW94]. In this way, a *type system* prevents runtime errors by detecting potentially erroneous incompatible method calls and assignments before the program is executed. More recently, *dependent type systems* introduce internal states of objects and explicitly capture how method calls transform the internal state via preconditions, postconditions, and invariants, to increase accuracy of the error prediction [AG26].

In the workflow modelling world, the situation is reversed. Activities have no signature, only an abstract name, and the internal state of a workflow, how it is affected by executing activities, and how it enables and disables them is in the forefront. We could say, that the analogy of a *type system* for workflows is a *soundness notion* [Aal+11]. A *soundness* check, like static type checking, accepts "well behaved" workflow models and rejects those

with potentially erroneous behaviour. We can check *soundness* of individual workflows in isolation, but it gets more complicated once interaction is introduced. Unlike in programming, where interaction is a single-line method call, by having explicit knowledge of the internal states, interaction in modelling is a coevolution of two or more workflows over multiple synchronized activity executions. This coevolution is formalized in *protocols* [Nie93] or *conversations* [Fah19], the workflows are often formalised as *modules*, and their composition is studied [FR24]. Composition of *sound* modules does not in general yield a *sound* system and so composition and construction rules and requirements are studied as well [Wer+22].

Furthermore, we must differentiate between *composition* and *interaction* [Des24]. Composition is a permanent glueing of two or more modules into a new module. Once composed, two objects effectively become one. Interaction is transient, an object may synchronise the same activity with different objects over multiple executions. Objects may also create longer-lasting connections (relations) to other objects that require they synchronise together as long as they are connected. Furthermore, synchronisation of activities between objects may not be 1-to-1 [Fah19]. The modelling, behaviour, and *soundness* of interacting workflows is one of the key research areas of *object-centric process mining* [GMW25; Wer+22].

If *type systems* are analogous to *soundness*, then *type inheritance* is analogous to *sound substitutability* in a modular system of interacting workflow processes. We specify communication protocols and interaction interfaces, to model modules that implement (inherit from) the interfaces, and whose interaction creates the full system. *Type inheritance* should allow us to substitute one module for another without breaking the system as long as they both implement the same interfaces. The goal in modelling and programming is ultimately the same. We formalize interaction interfaces in order to perform a static test that rejects potentially erroneous compositions and interactions.

The motivation for why the process-centric objects paradigm should support *type inheritance* is therefore clear. Just like an object-oriented program is an interconnected web of method calls, a process-centric objects system is an interconnected web of synchronized activity executions. We need to be able to formally specify communication protocols, their interfaces, and a testing procedure, that guarantees that if two workflow models correctly implement (inherit from) the same interface, then we can freely swap instances of one for instances of the other without breaking the system. *Type inheritance* is therefore inherently behavioural [BDP99].

Different perspectives on the behavioural aspect of *type inheritance* are studied in the literature by Basten and van der Aalst [AB97; Bv01], Nierstrasz [Nie93], and Balzarotti, De Cindio, and Pomello [BDP99]. The approach of Basten and van der Aalst is based on behavioural equivalence. They propose, that a subclass workflow model should be allowed to introduce new activities to the superclass's workflow model (an interface in our

context), as long as, after we silence (make unobservable) or block (effectively remove) the new activities, the subclass behaves identically to the superclass [AB97; Bv01].

Nierstrasz focuses on the type theoretical perspective and substitutability [Nie93; NP90]. In his setting a process-centric object A can be substituted for a process-centric object B iff the behaviour of B is a subset of the behaviour of A and additionally the behaviour that leads to a failure in A is a subset of the behaviour that leads to a failure in B , i.e. we can substitute A for B (A inherits from B) iff A can do whatever B could and A does not fail if we did not expect B to fail in the same situation [Nie93].

Balzarotti, De Cindio, and Pomello combine the approaches of Basten and van der Aalst, and Nierstrasz. They argue *type inheritance* based on a behavioural equivalence relations is too strict for many practical applications and, like Nierstrasz, prefer a preorder relation based on behavioural inclusion and failure inclusion. However, like Basten and van der Aalst, they permit the introduction of new internal silent activities in the subclass's workflow, as well as *renaming* of activities. Renaming means that an activity with some name in the subclass may play the role of an activity with a different name in the superclass [BDP99].

The literature studying inheritance of lifecycles, usually does not consider process models extended with data, as is the case in the process-centric objects paradigm. Workflows with data are studied in *object-centric process mining*, but only with respect to *soundness*, not inheritance. In the object-oriented world, activities in a workflow can be thought of as methods of an object and data associations to an activity can be thought of as method arguments. To ensure substitutability in the object-oriented world, a method of a subclass overwriting the same method in the superclass can change the argument types only contra-variantly, i.e. make them more general, and change the return type and the thrown exceptions only co-variantly, i.e. make them more specific [LW94]. It is reasonable to assume a similar rule should apply to process-centric objects as well. Activities do not have a return type nor exceptions, but they have data associations, which can be understood as method arguments. We should therefore under *type inheritance* be allowed to turn an existing read data association into a read/write association (contra-variant generalisation), but not vice versa.

Overall, *type inheritance* in the process-centric objects paradigm should support substitutability, modular design, and should aid in the prevention of runtime failures. It should allow us to specify communication protocols, be it 1-to-1, 1-to-many, or many-to-many, and to check if a workflow model implements one of the interfaces of a communication protocol correctly. In our opinion, we should look to behaviour based preorders along the lines of Balzarotti, De Cindio, and Pomello [BDP99], and Nierstrasz [Nie93], or to behavioral equivalences along the lines of Basten and van der Aalst [AB97; Bv01] to be the formal foundation of *type inheritance* for process-centric objects.

Chapter 5

Conclusion

Inheritance is an established modelling concept that organizes and structures object-oriented models and programs, yet it remains largely absent when designing and implementing model-driven workflow engines. This gap is particularly relevant for the process-centric objects paradigm, which combines executable process models with object-oriented structures, data, and interactions. For the process-centric objects paradigm, the literature and our practical experience suggest two complementary kinds of inheritance [BDP99]. *Implementation inheritance* is primarily structural and promotes code reuse and specialization of data and workflow models. *Type inheritance* is behavioural and promotes substitutability in a modular system of interacting process-centric objects [BDP99; NP90; Nie93]. We formulated requirements for both kinds of inheritance in the process-centric objects paradigm and believe the next steps should be a proof-of-concept implementation based on proposals from the existing literature.

We expect this implementation to reveal new research questions. For example, the approaches of Nierstrasz [Nie93], and Balzarotti, De Cindio, and Pomello [BDP99] assume that interaction between objects can be described by a regular language. While this assumption is plausible if objects interact 1-to-1, we suspect it might not cover 1-to-many and many-to-many interactions. If we need to keep track of the number of interaction partners in a 1-to-many interaction in order to wait for all of them, the structure reminds us of counting the number of opening and closing brackets in a context-free Dyck language.

Similarly, the computational complexity of checking if one process-centric workflow model inherits from another must be carefully considered. While decision procedures for some of the proposed inheritance notions on a restricted class of models exist [VB03], the underlying formalisms, like *strong bisimulation* or *failure equivalence*, are in general undecidable [Jan95; Hüt91]. We expect that, in order to provide software support for inheritance in an integrated development environment for the process-centric objects paradigm a tradeoff will need to be made between the computational complexity of checking the correctness of inheritance and the precision of the check, just as is the case in type checking of programming languages.

Bibliography

- [AB97] W. M. P. van der Aalst and T. Basten. “Life-cycle inheritance”. In: *Application and Theory of Petri Nets 1997*. Ed. by Pierre Azéma and Gianfranco Balbo. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 62–81. ISBN: 978-3-540-69187-7. DOI: 10.1007/3-540-63139-9_30.
- [Aal+11] W. M. P. van der Aalst et al. “Soundness of workflow nets: classification, decidability, and analysis”. In: *Form. Asp. Comput.* 23.3 (May 2011), pp. 333–363. ISSN: 0934-5043. DOI: 10.1007/s00165-010-0161-4.
- [AG26] Carlo Angiuli and Daniel Gratzer. *Principles of Dependent Type Theory*. Preprint, version 2026-03-21. Cambridge University Press, 2026. URL: www.danielgratzer.com/papers/type-theory-book.pdf. Forthcoming.
- [BDP99] Claudia Balzarotti, Fiorella De Cindio, and Lucia Pomello. “Observation Equivalences for the Semantics of Inheritance”. In: *Formal Methods for Open Object-Based Distributed Systems*. Ed. by Paolo Ciancarini, Alessandro Fantechi, and Robert Gorrieri. Boston, MA: Springer US, 1999, pp. 67–82. ISBN: 978-0-387-35562-7. DOI: 10.1007/978-0-387-35562-7_6.
- [Bv01] Twan Basten and Wil M.P. van der Aalst. “Inheritance of behavior”. In: *The Journal of Logic and Algebraic Programming* 47.2 (2001), pp. 47–145. ISSN: 1567-8326. DOI: 10.1016/S1567-8326(00)00004-7.
- [Des24] Jörg Desel. “Composing Chameleons”. In: *Joint Workshop Proceedings of PNSE, ATAED, and PeNGE at PETRI NETS 2025* (Geneva, Switzerland, June 23–24, 2025). CEUR Workshop Proceedings 3998. 2024, pp. 217–223. URL: <https://ceur-ws.org/Vol-3998/paper15.pdf>.
- [Fah19] Dirk Fahland. “Describing Behavior of Processes with Many-to-Many Interactions”. In: *Application and Theory of Petri Nets and Concurrency*. Ed. by Susanna Donatelli and Stefan Haar. Cham: Springer International Publishing, 2019, pp. 3–24. ISBN: 978-3-030-21571-2. DOI: 10.1007/978-3-030-21571-2_1.

- [FR24] Peter Fettke and Wolfgang Reisig. *Understanding the Digital World. Modeling with HERAKLIT*. Springer Cham, 2024. ISBN: 978-3-031-61898-7. DOI: 10.1007/978-3-031-61898-7.
- [GMW25] Alessandro Gianola, Marco Montali, and Sarah Winkler. “Object-Centric Processes with Structured Data and Exact Synchronization”. In: *Advanced Information Systems Engineering*. Ed. by John Krogstie et al. Cham: Springer Nature Switzerland, 2025, pp. 185–202. ISBN: 978-3-031-94571-7. DOI: 10.1007/978-3-031-94571-7_11.
- [GG01] Rob van Glabbeek and Ursula Goltz. “Refinement of actions and equivalence notions for concurrent systems”. In: *Acta Informatica* 37 (2001), pp. 229–327. DOI: 10.1007/s002360000041.
- [Hüt91] Hans Hüttel. “Decidability, behavioural equivalences and infinite transition graphs”. PhD thesis. University of Edinburgh, 1991.
- [Jan95] Petr Jančar. “Undecidability of bisimilarity for Petri nets and some related problems”. In: *Theoretical Computer Science* 148.2 (1995). Selected Papers of the Eleventh Symposium on Theoretical Aspects of Computer Science, pp. 281–301. ISSN: 0304-3975. DOI: 10.1016/0304-3975(95)00037-W.
- [JMP24] Gabriel Juhás, Milan Mladoniczky, and Luboš Petrovič. “Practical Experience with Petriflow: Enriched Process Models Serving as Implementation”. In: *Modellierung 2024 - Workshop Proceedings, Potsdam, Germany, March 12-15, 2024*. Ed. by Holger Giese and Kristina Rosenthal. Gesellschaft für Informatik e.V., 2024, p. 24. DOI: 10.18420/MODELLIERUNG2024-WS-024. URL: <https://doi.org/10.18420/modellierung2024-ws-024>.
- [Juh+22] Gabriel Juhás et al. “Low-code platforms and languages: the future of software development”. In: *2022 20th International Conference on Emerging eLearning Technologies and Applications (ICETA)* (2022), pp. 286–293. DOI: 10.1109/ICETA57911.2022.9974697. URL: <https://api.semanticscholar.org/CorpusID:254737546>.
- [Juh+24] Gabriel Juhás et al. “Can Low-Code Languages be Used for Core Enterprise Applications?” In: *2024 International Conference on Emerging eLearning Technologies and Applications (ICETA)*. 2024, pp. 232–242. DOI: 10.1109/ICETA63795.2024.10850790.
- [Juh+25a] Gabriel Juhás et al. “Petriflow: A Low-Code Language for Object Centric Processes”. In: *AWPN 2025 workshop proceedings*. Ed. by Robert Lorenz, Robin Bergenthum, and Patrizia Schalk. Bonn: Gesellschaft für Informatik e.V., 2025, pp. 3–22. DOI: 10.18420/AWPN2025-01.

- [Juh+25b] Gabriel Juhás et al. “Reuniting Data, Processes, and Forms: Petriflow Object-Centric Processes in the Age of AI”. In: *2025 International Conference on Emerging eLearning Technologies and Applications (ICETA)*. 2025, pp. 290–298. DOI: 10.1109/ICETA67772.2025.11280270.
- [Jur18] Milan Mladoniczky Juraj Mažári Gabriel Juhás. “Petriflow in Actions: Events Call Actions Call Events”. In: *Algorithms and Tools for Petri Nets* (2018), pp. 21–26. URL: <https://opus.bibliothek.uni-augsburg.de/opus4/frontdoor/deliver/index/docId/41861/file/awpn18-lorenz-metzger-OPUS.pdf>.
- [LW94] Barbara H. Liskov and Jeannette M. Wing. “A behavioral notion of subtyping”. In: *ACM Trans. Program. Lang. Syst.* 16.6 (Nov. 1994), pp. 1811–1841. ISSN: 0164-0925. DOI: 10.1145/197320.197383.
- [Nie93] Oscar Nierstrasz. “Regular types for active objects”. In: *SIGPLAN Not.* 28.10 (Oct. 1993), pp. 1–15. ISSN: 0362-1340. DOI: 10.1145/167962.167976.
- [NP90] Oscar Nierstrasz and Michael Papathomas. “Viewing object as patterns of communicating agents”. In: *SIGPLAN Not.* 25.10 (Sept. 1990), pp. 38–43. ISSN: 0362-1340. DOI: 10.1145/97946.97952. URL: <https://doi.org/10.1145/97946.97952>.
- [Tai96] Antero Taivalsaari. “On the notion of inheritance”. In: *ACM Comput. Surv.* 28.3 (Sept. 1996), pp. 438–479. ISSN: 0360-0300. DOI: 10.1145/243439.243441.
- [VB03] H. M. W. Verbeek and T. Basten. “Deciding Life-Cycle Inheritance on Petri Nets”. In: *Applications and Theory of Petri Nets 2003*. Ed. by Wil M. P. van der Aalst and Eike Best. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 44–63. ISBN: 978-3-540-44919-5. DOI: 10.1007/3-540-44919-1_7.
- [Wer+22] Jan Martijn E. M. van der Werf et al. “Data and Process Resonance”. In: *Application and Theory of Petri Nets and Concurrency*. Ed. by Luca Bernardinello and Laure Petrucci. Cham: Springer International Publishing, 2022, pp. 369–392. ISBN: 978-3-031-06653-5. DOI: 10.1007/978-3-031-06653-5_19.